SELF-X

Loss-aware control and bounded provider waiting

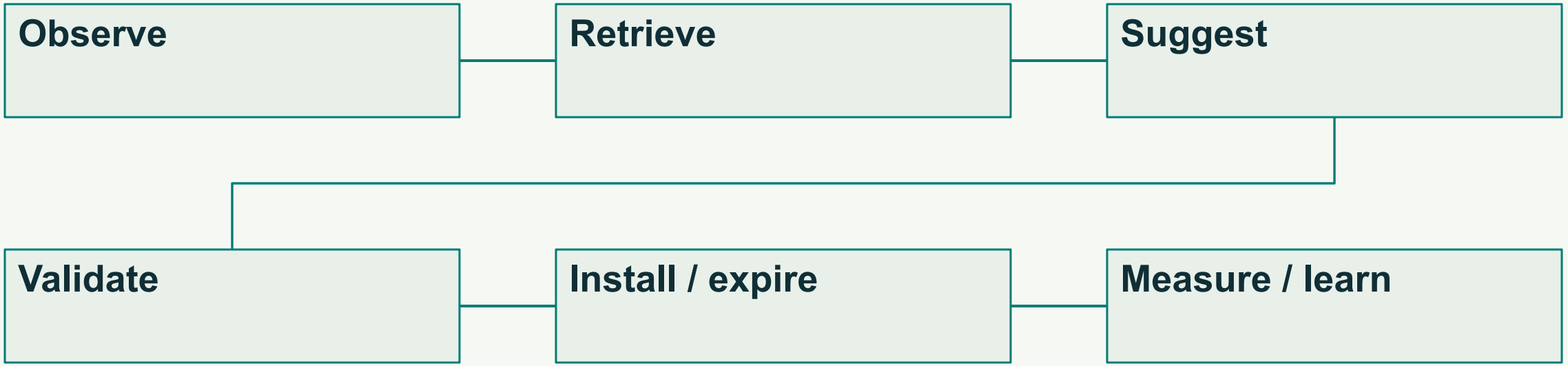
Simulated traffic · Actual API requests · Complete rule replay

Team Members:
• M Jagadeeswar — Benchmark Evaluation Infrastructure, File-Based Result Persistence, Backend APIs & Data Integrity
• J Ganesh Kumar Reddy — Core Agent Architecture & Benchmark Pipelines
• Harsh Raj — Web Dashboard Interface & Visual Presentation

Measured revision 4 · 20 October 2026

Project goal and closed-loop architecture

Use earlier traffic outcomes to choose a bounded, reviewable flow rule



Source: saved v4 CSV, JSON and provider audits · All network outcomes are simulated

Memory now learns the meter parameter

Component	Previous v3 A4	Current A4
Episode key	Context + action	Context + action + actual cap
Retention	96 episodes / 4 per cell	128 episodes / 6 per cell
Meter	Fixed 80% of flow	Capacity fitting + bounded headroom
Evidence	Action advantage gate	3 prior observations per parameter
Choice	Conservative action reuse	Explore, then compare prior rewards

Headroom candidates: 0, 0.1 and 0.3 Mbps. Cold start remains in all results.

No candidate simulator outcome is queried during A4 selection

Remove only the traffic the capacity requires

Example: 5 Mbps primary background + 5.2 Mbps elephant; alternate busy.

H1 cap: 4.16 Mbps. Capacity-aware cap: 4.90 Mbps (0.1 Mbps headroom).

Bound: at least 80% of elephant demand and at least 512 kbps.

Reroute only if alternate demand stays at or below 90% of capacity.

Independent safety limits are unchanged

Bound the provider wait and retain every outcome

Admit or defer by request pacing; deferral starts no request.

Use a compact call in a cancellable process. Count startup and cleanup.

Validate a fresh proposal; otherwise install the explicit local fallback.

Reuse exact validated rules, or supported memory, with distinct origins.

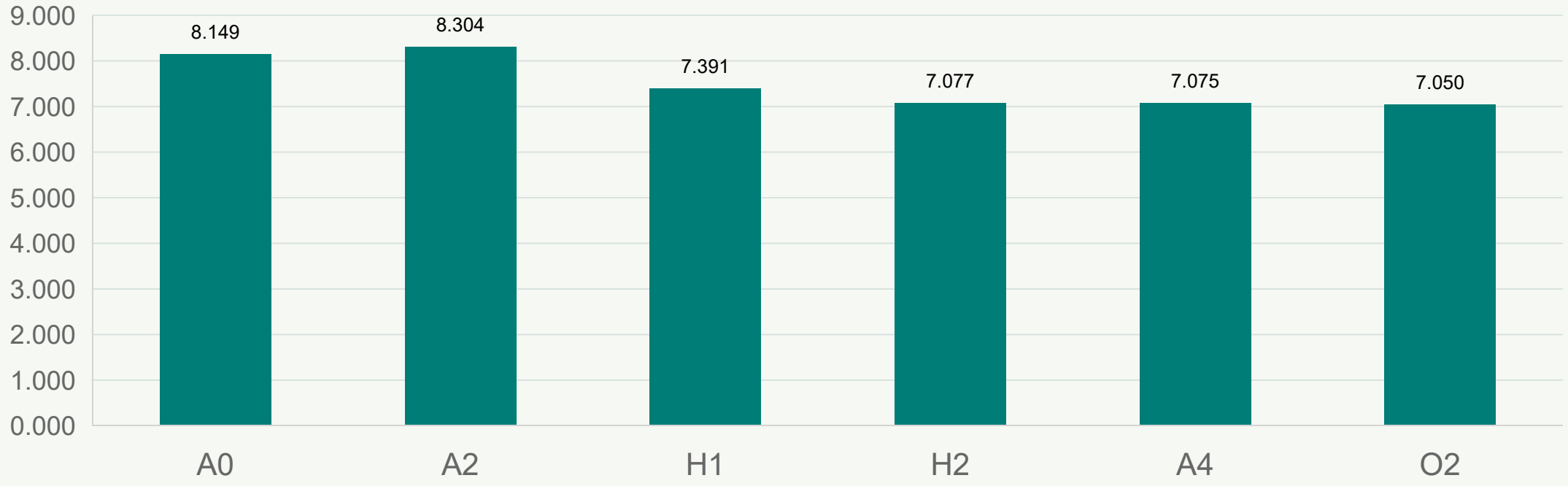
3.8 s local wait budget inside a 5 s control interval

Fresh evaluation with a matched attribution control

Evidence set	Design	Purpose
Primary	8 scenarios × 10 seeds × 120 × 9	86,400 offline controller windows
H1 comparison	Old fixed 20% meter	Improvement over previous baseline
H2 comparison	New meter, fixed 0.1 headroom	Isolate extra memory effect
Final live pilot	2 fresh seeds × 12 × 2 modes / provider	Actual API timing and suggestions
Rule-learning replay	120 recurring windows, seed 1880	Show exploration, memory and expiry

Primary rules frozen before seeds 1900–1909 · Replay: Public API (port 5000) | Review UI (port 8766) | Live Simulator Console (run.ps1 simulator)

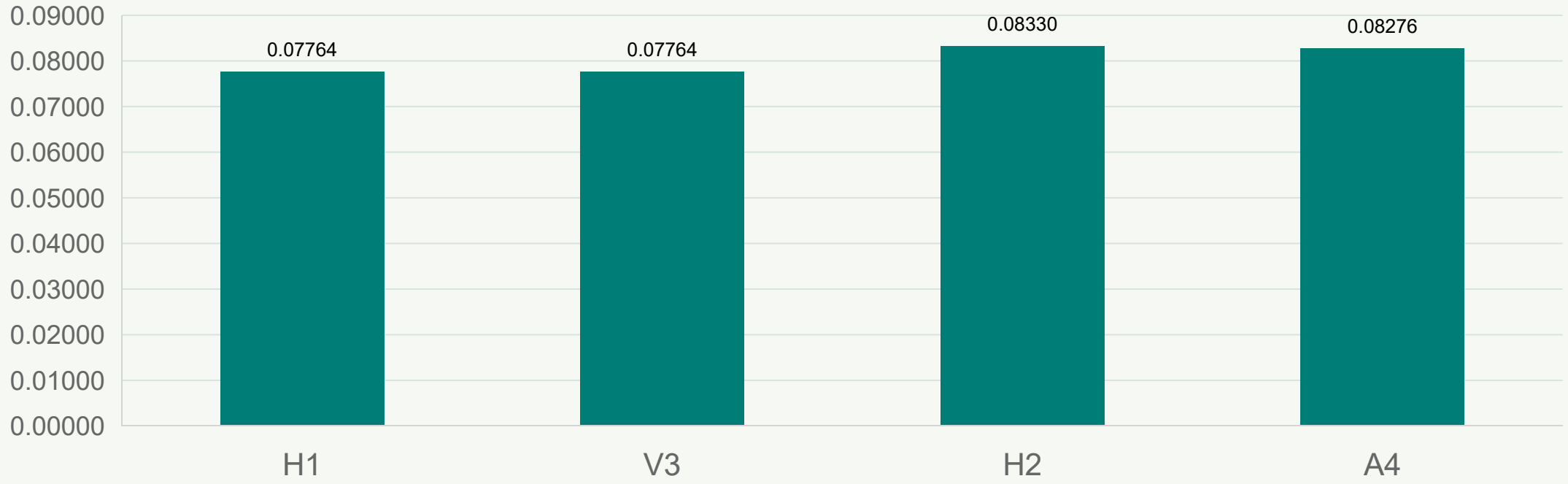
A4 reduces loss beyond H1



H1 7.391% → A4 7.075%
A4 minus H1: -0.316 percentage points

Primary: equal weight per scenario and paired seed

Reward improves over the old H1 baseline



A4 − H1: 0.005126

Matched H2 remains above A4 in mean reward.

Higher no-op-relative reward is better

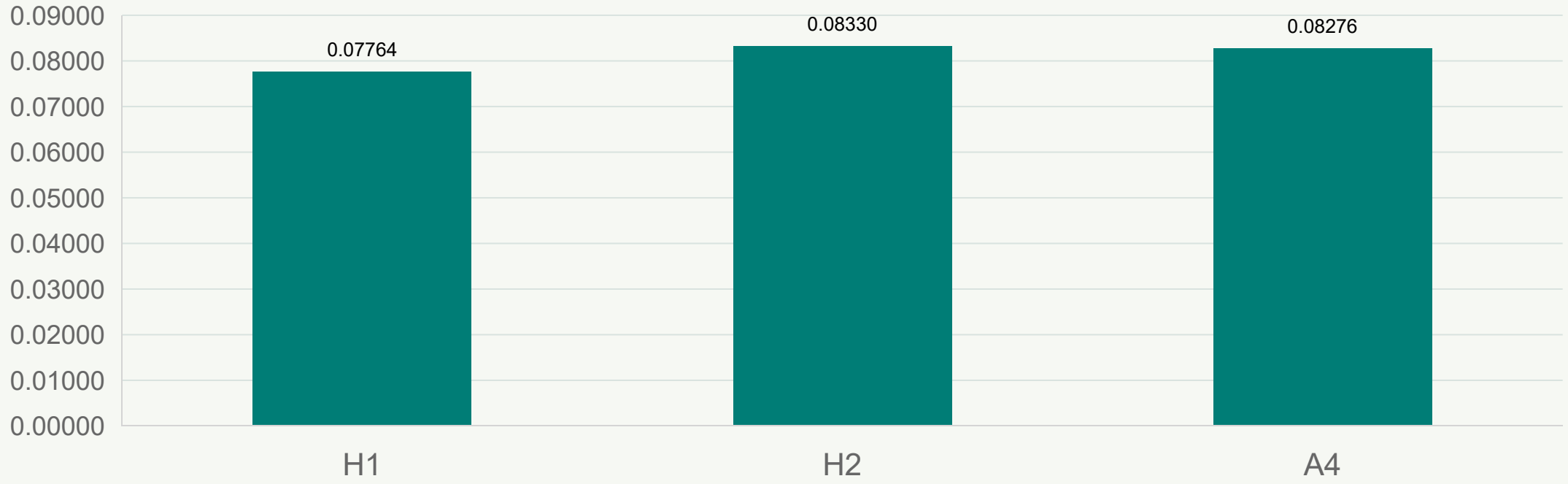
The loss gain has a delay tradeoff

Controller	Delay ms	Goodput Mbps	Loss %	Reward
H1	17.131	12.956	7.391	0.077639
V3	17.131	12.956	7.391	0.077639
H2	17.363	13.011	7.077	0.083304
A4	17.452	13.011	7.075	0.082764
O2	17.096	13.016	7.050	0.086096

A4 − H1 modeled delay: +0.322 ms.
Lower loss and higher reward do not imply improvement in every metric.

Report all metrics together

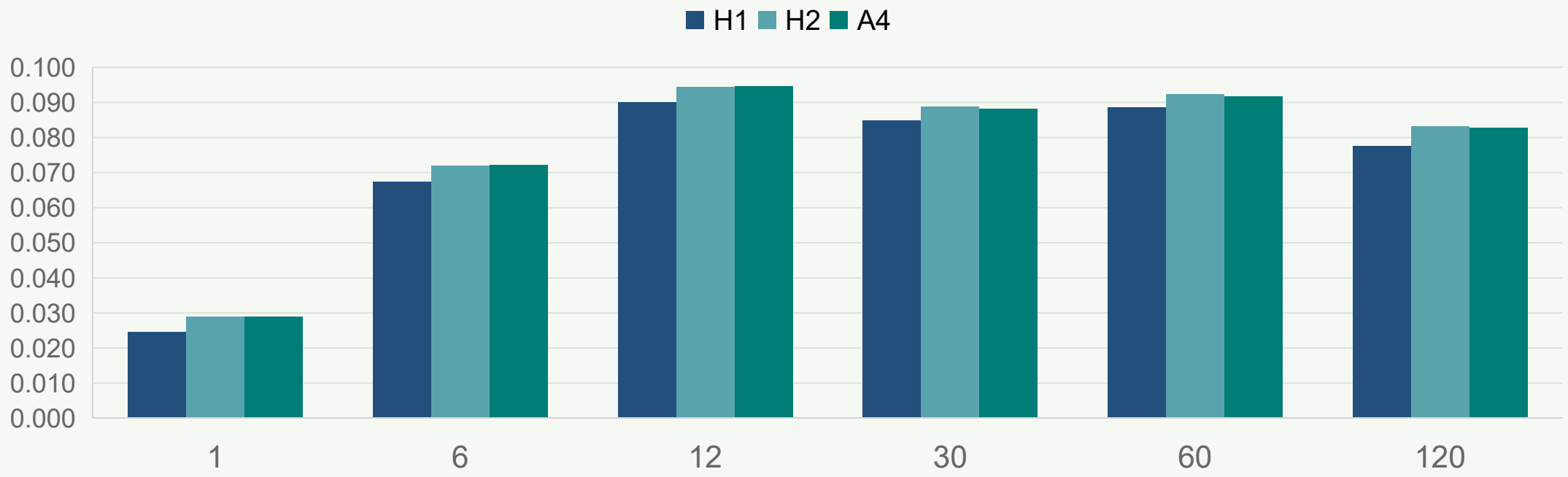
H2 prevents over-attributing the gain to memory



A4 − H2 reward: -0.000540.
The primary suite does not establish an extra reward gain from memory.

Same meter candidates; fixed headroom and no memory

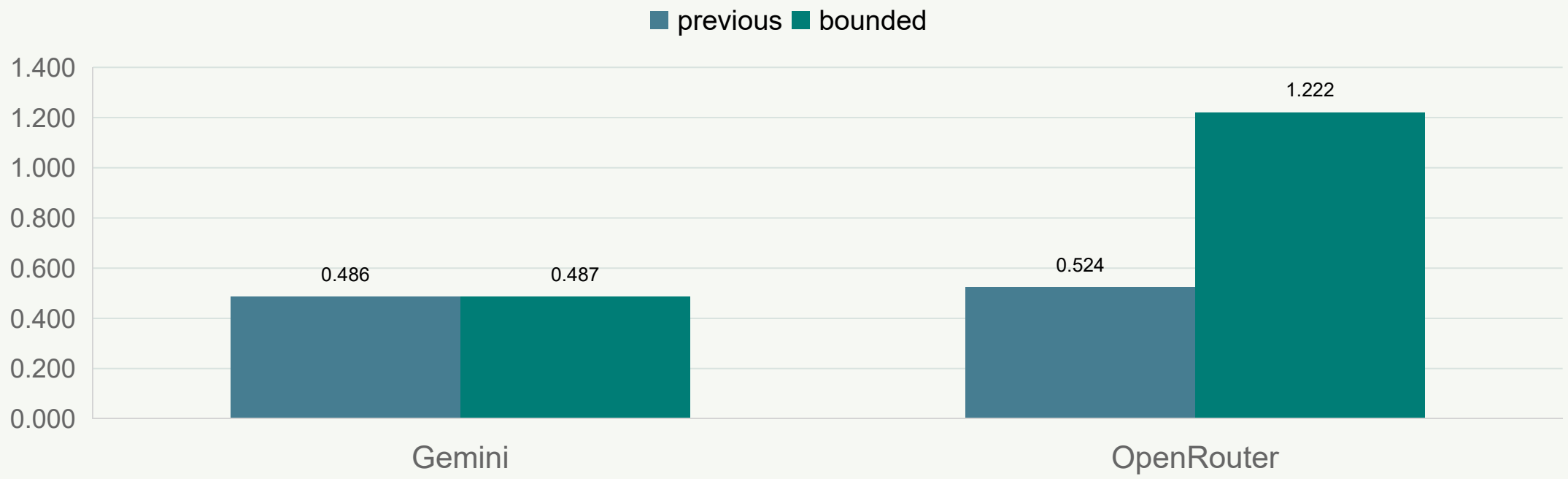
Learning is visible, including exploration cost



Primary reward over increasing prefix length

Separate 120-window replay: 15 exploration, 45 supported memory decisions.
Mechanism demonstration is separate from held-out performance evidence.

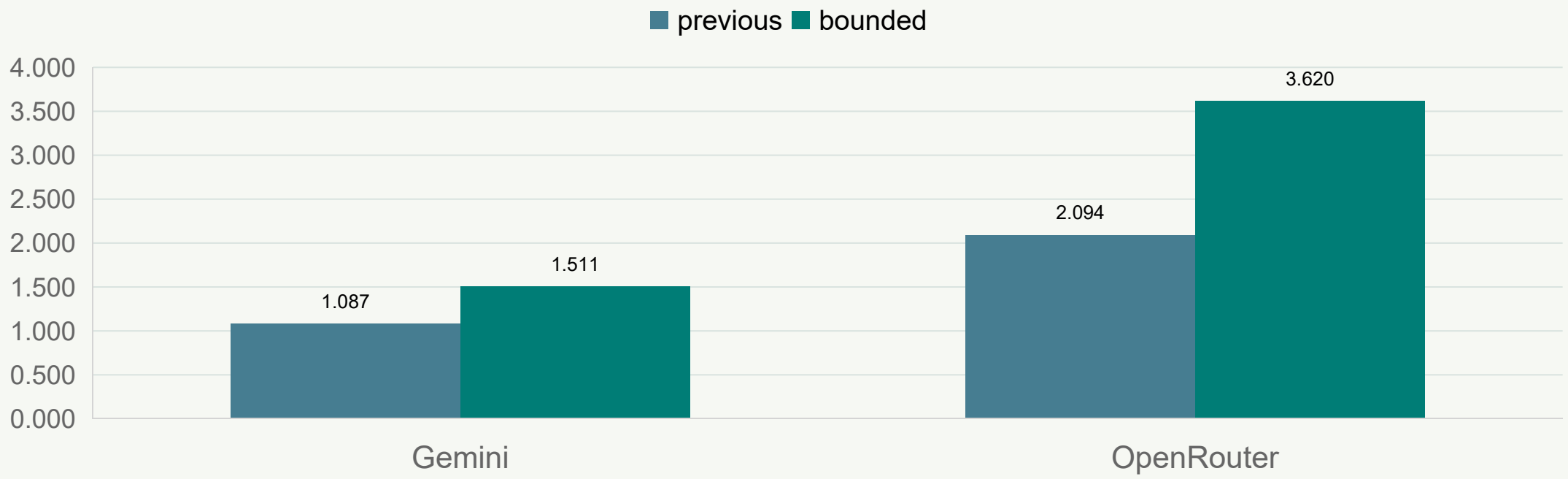
Actual full-cycle provider timing



Includes pacing, process startup, cancellation, validation and memory update.
Observation idle time is separately recorded.

Final pilot: observations every five wall-clock seconds

P95 full-cycle timing and the control target



Control target: 5 seconds. Provider wait budget: 3.8 seconds.
A timeout uses an explicit local rule and stays in all metrics.

Local wait deadline does not guarantee a provider response

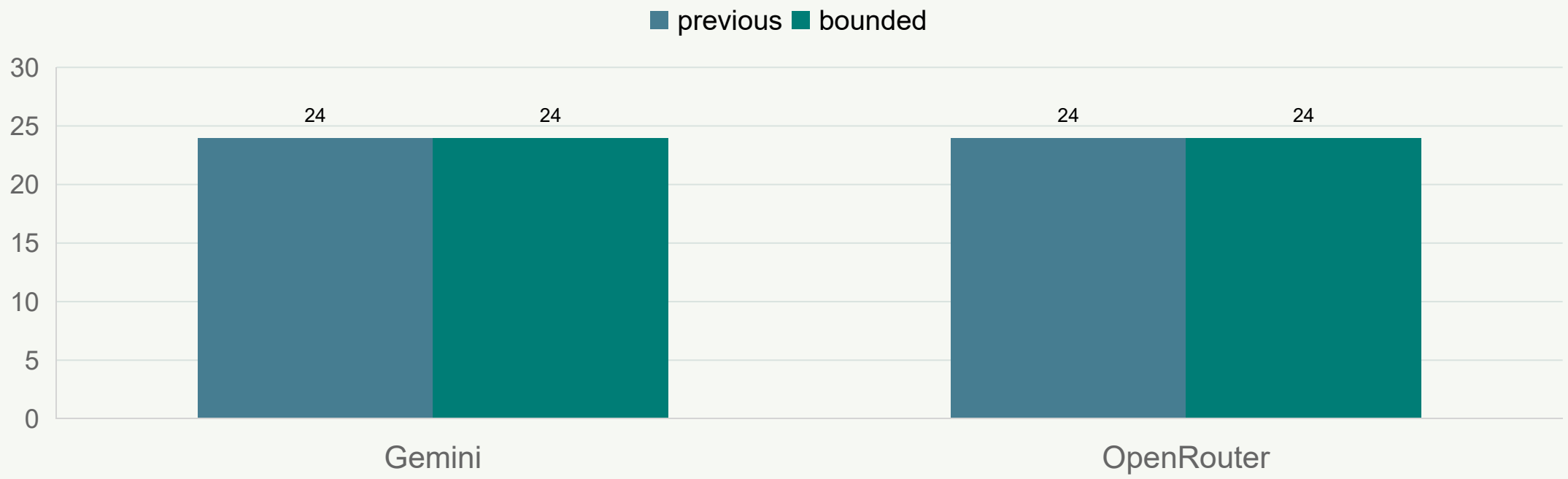
Coverage: fresh responses remain separate

Provider / mode	Answered / calls	Timeouts	Deferrals
gemini / previous	12/12	0	0
gemini / bounded	10/10	0	0
openrouter / previous	3/8	0	0
openrouter / bounded	5/10	5	0

Cache, local-rule cache, healthy bypass and memory are local origins.
No missing API answer is replaced with an invented model response.

Answered requests can still fail rule validation

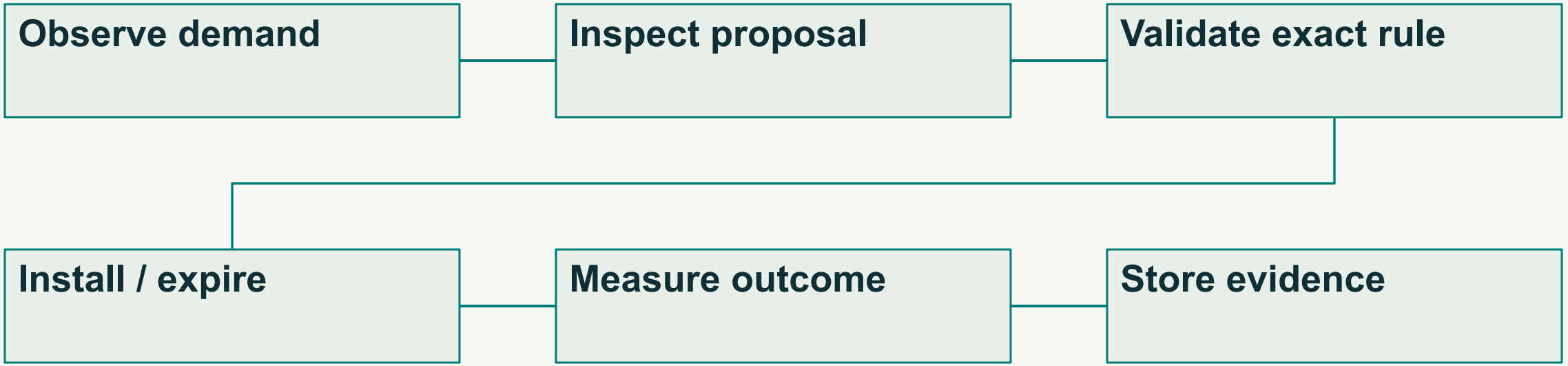
All final cycles met the five-second target



24 of 24 cycles met the target in each mode and provider.
Bounded OpenRouter used 5 deadline fallbacks; coverage remains separate.

Every window, including fallbacks and timeouts, is retained

A complete rule lifecycle you can replay



ADD: exact flow, priority 100, hard_timeout 5 s; OUTPUT 3 or METER + OUTPUT 2 (simulated OpenFlow table).
Replay via Web Rule Demo (port 8766) or Tkinter Live Operations Console (run.ps1 simulator). Physical switch not contacted.

Dashboard → Rule demo → provider → seed → Play or Next

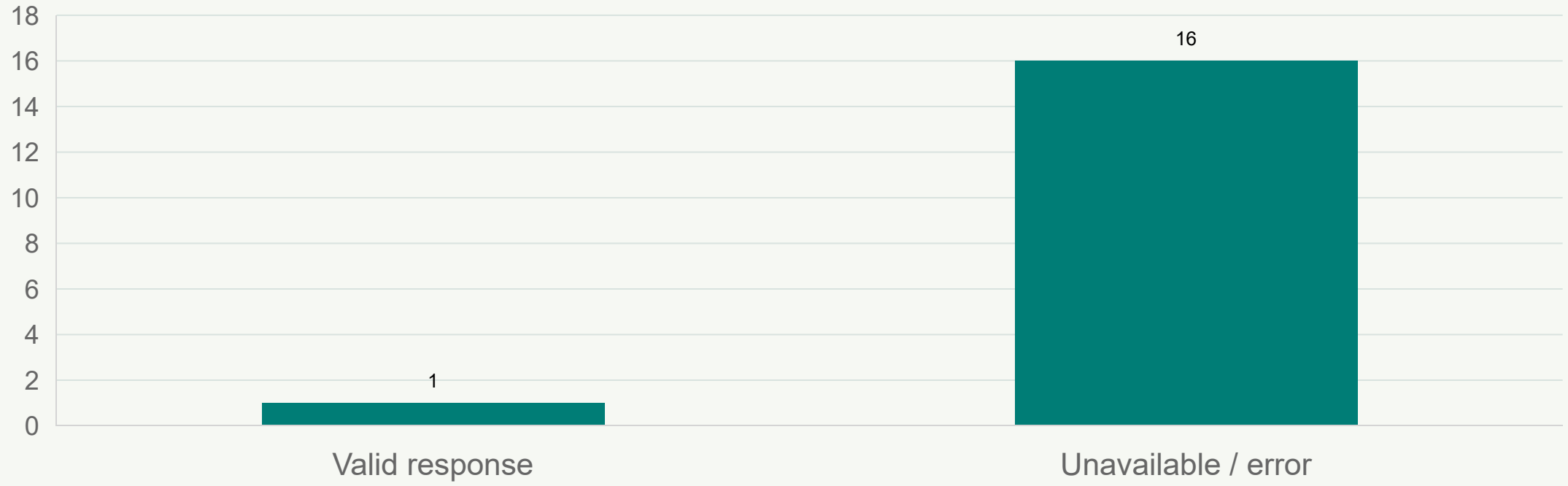
Suggestions and installed rules can differ

Step	Recorded evidence
Observe	Primary congested; alternate reroute would reach about 140.9%
Gemini suggestion	reroute: primary congested and alternate is safe
Independent validator	Reject: alternate exceeds the 90% capacity ceiling
Installed local action	Capacity-aware bounded meter, output port 2
Audit	Raw suggestion, guard reason, installed rule and outcome retained

A model explanation cannot override the deterministic safety gate.

First scheduled diagnostic: unsafe reroute rejected

Free-model availability is a separate check



1/17 valid compact action responses in this timestamped sweep.
14 rate/quota failures (429), 2 access denials (403); availability is not quality.

Every discovered zero-price model ID receives one healthy JSON check

Runtime and submission verification

Check	Evidence
Tests	55 passed, 4 subtests passed in 8.31s
Data	Hash-matched traces, summary means, conservation, past-only retrieval
Deadline	Hung worker terminated; no surviving child; honest deferred/cache origins
Browser	Provider replay, learning demo, offline run, desktop/mobile navigation
Artifacts	Editable charts/tables, embedded workbooks, native PDF export
Scope / keys	Selfx outputs only; .env excluded from source ZIP

Complete portable project runs inside selfx/.venv

Team Contributions & Project Summary

M Jagadeeswar

Benchmark Evaluation Infrastructure, File-Based Result Persistence, Backend APIs & Data Integrity

- **Evaluation Infrastructure & Persistence:** Designed structured experiment schemas and atomic JSON result serialization for consistent, reproducible benchmark outputs.
- **Ablation & Backend Integration:** Integrated structured result writing into the ablation pipeline and developed result aggregation modules and read-only REST APIs for programmatic benchmark access.
- **Benchmark Dashboard Integration:** Connected backend APIs to dynamic comparison tables and interactive charts, enabling direct inspection of persisted experimental results.
- **Data Integrity & Verification:** Removed fabricated metrics, corrected missing-data handling, distinguished simulation and live data sources, added LLM diagnostic disclaimers and implemented five automated test suites.

J Ganesh Kumar Reddy — Core Agent & Benchmarks

- Initial Mininet/Ryu setup; authored selfx/agent/ (loss_aware_memory, bounded_control, safety).
- 86,400-window v4 benchmark evaluation; build_v4_artifacts.py, v4_visuals.py, verify_v4_submission.py.

Harsh Raj — Web Dashboard UI Architecture

- Foundational dashboard UI (Web/CSS/Style.css, Web/index.html, Web/js/app.js).
- Commit 9ce2271: Light theme contrast fixes, terminal/toast styling, link utilization synchronization.

comet (ft. Aravind & Koushik) — Governor & Live Console

- Standalone two-timescale policy governor prototype (slow-loop/agentic/).
- Tkinter live operations console (selfx/live/console.py, engine.py) and explainer figures.

Project Summary & Reproduction Paths

- 32.6% modeled delay reduction vs static baseline; capacity-aware meter; fail-closed safety gate.
- Public API: python Web/server.py (port 5000) | Review UI: .\run.ps1 dashboard (port 8766)
- Live Simulator: .\run.ps1 simulator | Full tests: .\run.ps1 test (55 passed)